Upgrading old Java applets?

Terence Tao · 1 December 2008

https://terrytao.wordpress.com/2008/12/01/upgrading-old-java-applets/

tao2typ · converted to Typst 2026-10-14 (UTC)

This post is going to be of a technological nature rather than a mathematical one.

Several years ago (long before the advent of "Web 2.0"), I wrote a number of Java applets (in Java 1.0!) to illustrate various mathematical operations (e.g. Mobius transforms or honeycombs, the latter being a joint project with Allen Knutson), or to help my undergraduate students self-test their mathematical knowledge (via my multiple choice applet that tested them on several prepared quizzes). I had generally received quite positive feedback on these (the logic quiz on my multiple choice applet, in particular, seemed to be particularly good at identifying weaknesses in one's mathematical reasoning skills). However, being largely self-taught in my programming skills (beyond a handful of undergraduate CS classes), I did not manage to make my code easy to maintain or upgrade, and indeed I have not made any attempt to modernise these applets in any way for many years now.

And now it appears that these applets are slowly becoming incompatible with modern browsers. (IE7 still seems to run these applets well enough, but my version of Firefox no longer does (in fact, I should warn you that it actually freezes up on some of these applets), and only certain versions of Sun Java Virtual Machine seem to run the applets properly.) It also appears that the Java language itself has changed over the years; I have found that the most recent Java compilers have some minor issues with some of my old code.

But I would be interested in seeing some version of these applets (or at least the concepts behind these applets) to persist in some more stable and modern form (which presumably would mean using a different language than Java 1.0), as they seem to occupy a niche (viz., interactive demonstrations and testing of higher mathematical concepts) that does not appear to be well represented on the internet today. Given that I have much less time to devote to these things nowadays, I would be more than happy to somehow donate these applets to some collaborative or open source project that might be able to develop them in some better (and more Web 2.0 compatible) format, but I do not have much experience with these things and am not sure how to proceed. So I'm asking my technologically inclined readers if they have any suggestions for what to do with these old pieces of code. (I'm particularly interested in building up my multiple choice applet, as this seems well suited to some collaborative project in which multiple contributors can upload their own quizzes on various topics (which need not be restricted to mathematics) that could aid in students who wish to self-test their understanding of a given topic. And there are various features I would have liked – e.g. support for mathematical symbols – that I simply did not have the time or expertise to put into the applet, but which would have been very nice to have.) I would particularly prefer suggestions which might require some one-time work on my part, but not a continued obligation to maintain code indefinitely.

[Also, any suggestions for relatively quick fixes that would allow these applets to be runnable on most modern browsers without too much recoding on my part would be greatly appreciated.]

Comments (35)

Adam · 1 December 2008

Why not just create a project on google code, or even sourceforge, and let interested parties have access to the source to upgrade for you?

 **Mikael Vejdemo Johansson** · 1 December 2008

My suggestion for a very first try would be to see whether the applets will actually compile with Java 5.0 or 6.0. With any luck you can just specify origin language and things will magically work.
See for instance
http://www.particle.kth.se/~lindsey/JavaCourse/Book/Part1/Java/Chapter01/compatibility.html
for a discussion.

 **John** · 1 December 2008

I see a few options.
– You could use standardized web languages like XForms, SVG, and JavaScript, XHTML.
I would think the apps you are describing could be written using JavaScript which modifies SVG. Any book on SVG should have a section on manipulating SVG from JavaScript. JavaScript is a complete language and you can write some pretty complicated programs in JavaScript, so it should not be overlooked.
Some SVG books:
http://safari.oreilly.com/0672324296
http://safari.oreilly.com/0672322900
Or just search for SVG on oreilly or search for SVG on Amazon.
Another reference for this is the W3C working group
http://www.w3.org/2006/rwc/Activity.html
http://www.w3.org/2008/webapps/
http://www.w3.org/2008/webapps/wiki/Main_Page
although this is more work in progress type stuff.
The advantage is using standards that will be supported for a long time, the downside is there is a lot to learn.
– You could use Curl from the MIT research project
http://www.cag.lcs.mit.edu/curl/
http://en.wikipedia.org/wiki/Curl_programming_language
I haven't used it myself so I have no idea what the long term support would look like. Still, it is something to look into.
– There is the OpenLaszlo project
http://www.openlaszlo.org/
http://en.wikipedia.org/wiki/OpenLaszlo
In OpenLaszlo, you write the program and then it compiles it down to the JVM, to SWF (flash), or to straight HTML + JavaScript.
OpenLaszlo seems like the most promising, because it is an open source framework that compiles down to whatever is the latest web technology. It seems like if some newfangled web tech comes along, OpenLaszlo could (will) be modified to compile to it. Also, OpenLaszlo can compile to flash or straight DHTML (which means, just HTML plus JavaScript…).
– Lastly, you could use a closed library/platform like Microsoft's Silverlight, Adobe's Flash. Or you could use Java, just the latest versions. These options seem like the worst for long term portability.

 **Robert** · 1 December 2008

The multiple choice thingy does not require to run on the client side, that could be done on the server side in terms of a CGI script. I estimate it would take me 2-3 hours to rewrite it as a perl programm that reads the txt and presents the quiz. I would be willing to provide this time. Let me know if interested (helling@atdotde.de)

 Kit · 1 December 2008

I think an alternative way to develop multiple choice tool is to use the server based platform such as PHP, the benefits are obvious: no matter what is working on the background, as long as users can view html files (users need not to install Java VM since all the things are done in server), there is not compatible problem; it's very easy to add/del/view questions in database; it's possible to get some statistics informations like the correctness of each problem, the average time to solve a problem. The disadvantage is: we need a server. If you are interested in this, maybe I can help.

 **Aaron** · 1 December 2008

In my experience there tend to be alot of online interactive demos demonstrating concepts from computer science but few for mathematical concepts. I do recall seeing some videos on youtube that did a great job of showing Mobius transformations and the Riemann sphere.
I think you will find that there will be many people that would be able to help you with these java applets if you place the code online(I will second the suggestion of google code as the hosting site).
As Robert above suggested, building simple applications like this often only takes an afternoon, but I think that the more ambitious idea of a general framework for building these sort of things is an interesting idea.

 **Anonymous** · 2 December 2008

If the answers you get here don't satisfy you, slashdot.org would be a good place to ask this kind of question.

 **user** · 2 December 2008

The quickfix is of course to try to recompile it, as Mikael suggests. In the long run, however, it is my opinion that Adobe Flash (or Adobe AIR) is the way to go for anything interactive like the Möbius or Honeycomb demos. It's installed "everywhere" and while ActionScript 3 isn't a "real" language like Java, it's actually quite good for small stuff like this. Developing for Adobe Flash requires commercial software (although I managed to finish the applet I was creating within the first weekend of the 30 day demo), but Adobe AIR is free.
As for multiple choice: it depends on whether "cheating" users are an issue. If the answer doesn't have to be kept secret until the user answers the question, then I'd implement it all client-side using JavaScript, optionally communicating with the server using Ajax to load questions and answers or return results. If it should be kept secret, then server-side is the only way to go, but you could still use Ajax for instant feedback to give it a "Web 2.0-feel". One could also write something in Flash for this, but the only reason for doing so that I can think of is that it would allow the user to interactively fetch QA-packs from other sites (without having to navigate to the site in question).

 **Reid** · 2 December 2008

Well, there are two alternatives. Either fix the errors in the code and recompile it so that it'll run on today's browsers (not really web 2.0) or make it a web app (either standard or the buzz word web 2.0). But, if you want future maintainability, then it doesn't really matter what you do as languages and byte-compiled form specifications do always change over time. The only thing that is the solution for this is maintaining the apps.
All that being said, if one doesn't want require people to have Java installed or needed (yes, there are good reasons for this), then PHP plus a Database (txt?!?!? seriously Robert?!?!?) would be a good solution. This way, the browser only needs to understand HTML/etc. Something which most browser do (IE arguably being an exception ;)).
A warning though: Just putting code up and asking for someone to do something with it on sourceforge/etc is just about as guaranteed a death sentence for a project as one can get. People just

don't pay attention to that sort of thing. Your best bet is to get an offer for developing something from someone here or getting a grad student to do it or asking a colleague from the CS department if (s)he would do it or if they have a grad student that would. I'd strongly recommend you to get someone you know to do it, or someone that you know/trust to get someone to do it. Because, if you don't, your requirement of quality is in jeopardy. There's too many people out there that think they're good, but really *really* aren't.
My suggestion would be PHP and MySQL (or PostgreSQL if you prefer) with mimetex.cgi for displaying Math. The images from mimetex.cgi could be cached upon question final entry to improve the applications performance. This could be done using standard or web2.0 techniques. IMO, web2.0 adds unnecessary complexity (for at least some applications) and will cause problems on some browsers due to its reliance on Javascript which isn't as cross-platform as people would like to believe. It's unnecessary "window dressing," especially for these purposes.
But, no matter who does it, and I can't stress this enough, they *must* document it! Most people don't do this (or do it exceedingly poorly) and it makes it utter hell to maintain something, even for the person who wrote it. I'd recommend getting them to document the code well (no acronyms and full paragraph explanations of what does what) as well as a basic overview walk through document. That might seem like overkill, but it takes around 3 weeks for code to start to seem like it was written by someone else.
Good luck!



konradswanepoel · 2 December 2008

Speaking of Java applets that demonstrate mathematical (and physical) concepts, Greg Egan has a nice collection:
http://gregegan.customer.netspace.net.au/



antal · 2 December 2008

Upload the source code here and I will fix it. I wouldn't recommend re-coding it again in php… A dirty language in which (string)"false" == (int)0 is true is not for 1st class math guys. ;-)



Robert · 2 December 2008

I only mentioned txt since the file provided http://www.math.ucla.edu/~tao/java/MultipleChoice/logic.txt
is txt. And with perl it is not much of a pain to parse this.



NotATechGuy · 2 December 2008

What about making a contest out it?
And have your blog readers vote for the winner by texting…



Reid · 3 December 2008

@antal:
How arrogant of you to say.
I'm also wondering if you realise that there is a difference between not liking something and it being actually bad.
@Robert:
Fair enough. But, he did say that he wanted it done better for the future.



Terence Tao · 3 December 2008

Dear all,
Thanks for all the comments and suggestions!

Regarding the immediate question of compiling the applets, it does in fact seem that the applets work fine for the majority of browsers (my own copy of firefox doesn´t work, and I´ve received occasional complaints from others, but three other copies on three different computers that I tested do, as do four different copies of IE), and judging from the comments here, it seems that others were able to access the applets also. My unix copy of javac also managed to recompile the applets (after complaining a bit), so my concerns about incompatibility are perhaps premature.
In the longer term, I had in mind some sort of collaborative site (something like a wiki, perhaps) where contributors could upload or edit quizzes in a wide variety of topics (no reason to restrict it to mathematics, really), with some common front-end for anyone to self-test on whichever topics they wish (and I don´t think cheating is not a concern here – why would one want to cheat on a self-examination?). This is one reason why I made the format for the quizzes as plain as possible, to encourage others to write their own (though I only ever got one or two questions contributed). For this basic sort of application it seems a plain implementation is better than a fancy one (though I still would like maths symbol support). A server side solution would probably be better (Id want to make it as easy on the user as possible), though I have no idea how difficult it would be to maintain such a server (I would want something as low maintainance as possible). (For security reasons, I don´t think I´d be able to host something like that on the UCLA math department machines.) I will have to think some more (and talk to some technically minded people) to see what the best options are here.
I would also like to modernise my other applets at some point but it seems that the multiple choice one may be easier to implement and have a better return on investment, and so I would probably focus on it first.



Flea# · 3 December 2008

Professor Tao,
I think it would be a wonderful idea to make your software open source. I would recommend hosting your project with Google Code, which can be found here: http://code.google.com/hosting/ You can create up to 10 open source hosted projects for free and then people who are interested can work on the projects and keep them maintained.
Regards,
Flea#



damidami · 3 December 2008

Dear Terence,
I ported your multiple choice applet into php, you can view the results here
http://www.resueltosya.com/quiz/
You can choose all the quizzes that were on the original applet, plus a short demo I made that uses latex, images, and videos.
It's a first draft, it doen't have a stylesheet so it looks rather ugly.
Any feedback is appreciated. I can send you the code if you want it.
Don't worry if for security reasons, you can't host it on the UCLA, you can leave it in this host if you want.
To be done:
.score system (I hadn't done it yet)
.stylesheets
.more testing
.upload of new quizzes.
.tags for making easier the inclusion of tex
and lots of things more I am planning to do if I find the time.

Regards,
Damián.

 **Terence Tao** · 5 December 2008

Dear Damián,
The prototype looks great! I'm traveling at the moment and am not able to reply in detail yet, but this new format does have plenty of potential for the type of expansion I had in mind.

Top Posts « WordPress.com · 3 December 2008
[…] Upgrading old Java applets? This post is going to be of a technological nature rather than a mathematical one. Several years ago (long before the […] […]

 **anonymous** · 4 December 2008

The work done by damidami is great.
One small suggestion: it would be nice to be able to go back to the main page in one click. Right now I need to click Back many times to get back to the main page.

 **kt** · 4 December 2008

damidami: 1 suggestion: give vertical some spacing between the bottom 2 buttons and the radio buttons.

 **damidami** · 5 December 2008

Hi,
Anonymous: I have now added two buttons at the start of the page to restart the quiz or select another quiz.
kt: I have now changed the radio buttons to simple buttons in a table, so it looks more similar to the original applet.
I am parsing the original .txt files, so adding media as images, TeX, and videos, was simply a task of adding hyperlinks on the .txt
More suggestions are welcome.

 **Aditya Prasad** · 6 December 2008

Sorry if this is off topic, but while browsing logic.txt, I stumbled upon this:
Question. Suppose one wishes to prove that "if all X are Y, then all Z are W". To do this, it would suffice to show that:
…
All Y are X, and all Z are W.
…
But this is given as an *incorrect* answer.

 **Terence Tao** · 6 December 2008

Ah, I think I intended to have "All Y are X, and all W are Z" in that answer. Well spotted! (This is one reason, by the way, why I want to move the quizzes to a more wiki-like format, so that they can be corrected and updated more easily.)

 **user** · 7 December 2008

As I wrote earlier, I think client-side JavaScript is the way to go for the quizzes, assuming "cheaters" aren't an issue. The reason for this is that, excluding the scoring, everything can be implemented as a plain old web page. Moreover, such pages can be kept in a wiki, with support for equations, inline

images (for geometric figures, algebra, ...) or whatever already taken care of. All that's required is then to add some JavaScript to get the scoring working.
To make it more concrete, I went ahead and wrote the necessary JavaScript-glue, added two of the quizzes (logic and continuity) and put it all in a wiki: http://scherk.pbwiki.com/ (to edit you'll need an account, but registering is really simple and requires no personal information). The equation support isn't perfect: the font size is a bit too large, and there are some alignment problems, but this is more of a PBwiki thing, not related to the concept itself.
I agree with Reid that JavaScript isn't that cross-platform, in particular not when it comes to interacting with the DOM (and I'm sure there are bugs in the code I've written!). However, should a new browser version be incompatible with the code, all that will happen is that the scoring stops working, which is not a big deal.
Again, there may be bugs or other mistakes, but hopefully you'll get the idea of what a client-side solution can look like.
(Sidenote: some silly people actually cheat on self-tests because it makes them feel good up until they day the actually have to take the real exam!)



damidami · 8 December 2008

user:
I liked your approach, I also thinked about an 'all client' approach using javascript at first, because it is a bit faster, althought it has the disadvantage of been able to cheat by viewing the html source code.
I also thinked that maybe in the future it would be nice if the client could log into the quiz and the server could remember the results he already made.
And for statistical purposes it would be nice if the server could see what the client has responded to a quiz.
What I liked most of your approach is the ability to edit the quiz like a wiki. (I hadn't tested it but it surely works)
What is missing in my approach is the ability to edit the quiz .txt, I had searched for wiki-like editors for text files on the internet, but couldn't find any yet.
I know my approach is slower because it is server side, and by now it isn't using ajax so it reloads the whole page with each submit, but that could change in the future.
Are you parsing the original .txt files? I think we should all agree on the format for the quizzes first, so we can then have multiple implementations of the quizzes, but all of them should work on any implementation.
Terence: Would you want to keep the same format for the txt? Or maybe an xml approach would be nice? The txt has the advantage that anyone can edit it easily with a text editor, but the xml is not much more difficult, and it is a bit more web-oriented I think, as it has to have hyperlinks to images and videos now, and the txt doesn't suggest you can use html tags there.
For example I found an xml quiz format here: http://www.director-online.com/buildArticle.php?id=1012



Terence Tao · 9 December 2008

Dear user:
The javascript solution is quite nice; the mouse clicking is cut down to the bare minimum (one per question) and there is no reloading between questions. Of course, as long as the quizzes are in a public, open, and standardised format, one can have any number of front ends for them (for instance, damidami's server-side quizzes could presumably use the wiki files for their data, thus ensuring that they always stay current). So I guess one does have to settle on a fixed format now before moving much further on this.

I am not particularly attached to my original txt format, though I am glad that it was sufficiently human readable to have it converted twice now without much supporting documentation. I had a few little extra parameters in the format, such as a scoring system and the ability to shuffle questions around, but these are not particularly necessary (and may as well be put on the front ends rather than in the quizzes itself, anyway). I am going to presume that the intended users here are mature enough to use this as a self-testing tool and to avoid cheating; I'm not sure any quiz of this form would be helpful to a student who would cheat even on self-tested examinations. Having long-term statistics for students to see their score improve over time would be a nice feature, but can presumably always be implemented later, once the basic data format is fixed.
I like the wiki format, being reasonably familiar to use (thanks to Wikipedia); I don't know of any other format which allows easy and familiar editing by the casual user. The format that user has chosen seems to be quite clean and one could adopt it as the standard (though one would have to provide some minimal documentation for it, I guess). I have perhaps two concerns though. The first is what happens if one makes an edit that breaks the format – presumably the wiki software would allow it, but the javascript should complain immediately about the first line that didn't compile properly. (Since edits on wiki pages can be easily undone, this is probably not too much of a problem.) The other is to make it as easy as possible to start a new quiz in the same format as the existing ones. Having the pre-made templates is probably a good short-term solution for now, but ideally one could press a "Create new quiz" button, fill out a short form for the basic parameters (e.g. quiz title, expected number of questions, whether one expects to have partially correct answers) and then generate a template for that too.
I don't see a "discussion" page attached to each wiki page, as in Wikipedia, but I assume that one could attach one (or some equivalent). Certainly there needs to be a place to put commentary about each quiz (e.g. the motivation of the quiz, level of background expected, etc.) that won't be compiled into the quiz itself.
How easily could the wiki quiz support LaTeX, HTML, images, etc? In some cases one would want this in the answers and commentary as well as in the quiz question itself. [*Update, Dec 9:* Ah, I see the continuity quiz already exhibits LaTeX in the question, answers, and commentary. Excellent!]
Incidentally, in some of my quizzes there are multiple correct answers, or one correct and one partially correct answer, etc. I would very much like to retain this feature – the restriction of having only one correct answer to a given multiple choice question is artificial, and gives a misleading impression as to how real mathematical problems are solved. (In principle one could also give questions with no correct answers provided, though this seems somewhat sadistic.)
I plan to write a post here soon explaining my pedagogical position behind these quizzes, and why I feel it would be good to have a repository of various such quizzes (initially in mathematics, but perhaps later in other subjects also) for self-testing.



Reid · 9 December 2008

@user:
Um, scoring not working not being a big deal?!?!? Is that not the purpose of the quiz? Also, have you tested that code on all major browsers on all the platforms that they work on? Because, I know that there have been issues with browsers that only existed on one platform, but not the other. As in, just because it might run on Safari on Windows, doesn't mean the same on OS X. etc. This includes, but goes beyond the DOM as well.
It doesn't really make sense to make maintenance more of a bother than it should be. Especially, when the purpose is for this to just work. In other words, ignoring the problems what *will* arise just because you like the solution isn't exactly the wisest course of action.
On another note, I think that people are jumping on the wiki band wagon where it isn't necessarily a good idea. In other words, just because one aspect of wiki's goes along with what one wants, doesn't

mean that other aspects of the wiki doesn't work against what one wants. For instance, defacement is a known issue and thinking "that won't happen here because it's academic" is naive. Some people just deface things because it's "fun." Or someone could fix a presumed mistake with all good intentions, but not realise that it wasn't a mistake and they actually broke something. That would actually hurt the quiz as it would confuse students. etc.
Regarding the format: Text files are just about as portable as it comes. Disregarding line ends of course. But, doing this by text files can get a bit complicated because of parsing. Because of this (and another point below), I still think that a DB is the way to go.
That being said, there are other alternatives to a full blown DBMS such as MySQL. SQLite is one. It also produces cross-platform DB files. But, at the same time, having any sort of direct file access is going to cost dearly in performance. As in, constantly opening and closing files is expensive. So, if the site is gets busy, things will bottleneck. This is not the case with a DBMS. Which is one of the major reasons why web sites (any one worth its salt) are DB driven, not text file driven. Though there are certainly other considerations there.
All in all, if we want to make something that "just works" there are not exactly many options. HTML is something that can be depended on and CSS to a somewhat lesser extent (sad isn't it) client-side. Other than that, nothing else is guaranteed. But, server-side, we can control everything (hopefully – IT departments can be irritating). So, we get choice of language, choice of DB, choice of web server, choice of… you get the idea. All that translates into a far more stable app. Because, everything is a known issue.
Remember, we're talking about stability and ease of maintenance. That is independent of what we like. Not every tool is meant to do everything.

 **Terence Tao** · 9 December 2008

Dear Reid,
Thanks for the comments. It occurs to me that we may in fact have stumbled on a best-of-both-worlds solution: one can use the wiki model for developing the quizzes (with the javascript being used primarily to test that the quiz questions work as intended), and the server-side model for storing stable versions of these quizzes (scraped from the wiki at periodic intervals, after new quizzes there have moved to a stable form) and delivering them to the student. In other words, development and delivery are largely decoupled (and the latter should be very low maintenance, once the initial coding, testing, and debugging is complete). I do want to make the barriers to contributing new problems or quizzes as low as possible, which is why I feel a wiki-type solution for development is preferable to an invitation-only collaboration. (Though one can always adjust the registration requirements of contributors of the wiki later if this turns out to be necessary.)
Incidentally, I view the scoring of these quizzes as of secondary importance only, and should not be a prominent part of the presentation of these quizzes. My intention here is not to create replicas of actual multiple choice examinations to use as practice runs – there are already several sites on the web that offer these sorts of things already (online practice SATs, etc.). Thus, for instance, I do not envision these quizzes being timed or otherwise presented in some simulation of a classroom examination format. Instead, the point is to identify – with instant feedback – specific strengths and weaknesses in one's knowledge of a subject. So getting the immediate "Correct", "Partially correct", or "Incorrect" response to each question as it is answered – together with a discussion of the misconceptions that could lead to a stated incorrect answer – would more valuable than the final score at the end of a quiz. (As I said before, I plan to make a more extended post on these pedagogical matters soon.)

 **user** · 10 December 2008

@damidami:

Right now I parse the text files and semi-automatically emit the required html (this is a one-time thing, done offline). Having a unified format for the questions sounds like a good idea, one could even have a quiz2latex-tool to make beautiful print-to-paper quizzes. A problem would be how to edit them. Currently the wiki uses plain old html to structure the quiz: one OL with id "questions" having one LI per question, with the answers kept in a second OL. The correct/incorrect/partially correct answers, and optionally comments, are marked by adding an UL to the answer in question. This makes it quite easy to edit the quizzes, although some care is necessary not to break the structure, but it's not the ideal exchange format. If the backend format was, say, XML (with external references for images?) we could still have different views — client based, server based, output to LaTeX etc — but editing would probably be more complicated.
Tao's suggestion to use the wiki as a "good enough" exchange format, and parse it into something cleaner (like XML) on regular intervals is perhaps the best solution within reach. At least it makes WYSIWYG editing easy, which is good thing if we want to make it easy to contribute new questions. As for statistics, it could probably be done client side (storing results in a cookie or a database, if the browser supports it). However, I think such a thing would be better suited for something server side, along the lines of what you have started. The main reason for this is that it's far easier to get long term storage server side, and it makes it easier to keep track of questions that change.
@Tao:
Thanks for your comments. I could add some code that "tests" the quiz. Perhaps it would be enough to add a link (in the footer) that when clicked automatically clicks every radio button, allowing the user see everything "expanded"? You're right about the need for discussion pages, PBwiki has no support built-in support for them, but creating an extra page per quiz and adding a "Discussion"-link to every quiz should be easy enough.
The code puts no requirement on the number of correct/incorrect/partially correct answers, but for scoring purposes it is currently assumed that there is at least one correct answer per question. Without a "next question" button this is essentially required, otherwise it's impossible to distinguish between "forgot to answer" (0 pt) and "correctly identified that there is no correct answer" (10 pt). One could automatically add a "None of the above"-alternative to questions without a correct answer, but it's probably just as easy to do so manually.
Also, an important question: Have you thought about how the quizzes should be licensed? Since you're the author of the (starting) material it's up to you to decide. A Creative Commons license (http://creativecommons.org/about/license/) might be a good idea, but you have to pick the flavor, perhaps by-nc-sa would be a good choice? Or will attribution be too complicated with multiple contributors? Do we want to allow commercial use? CC is not a good option for the JavaScript though, but I'd be happy to put that under BSD/GPL/whatever.
@Reid:
Obviously you and I have very different views of this project, so let me take a moment to explain my point of view. In my opinion the greatest threat to a project like this is that the people behind it lose interest in it. Maybe not in a year or two, but after ten or maybe twenty years people will have changed careers, have retired, be bogged down by administrative tasks and have no time for it, etc. For this reason I wanted a dead simple format for the questions and answers: a plain old web page. (A text file would still require some markup if we are to allow equations, images and such.) If we, or the Wayback machine, or someone else, manage to keep just one mirror of those files online — and mirroring a set of html documents and images is as easy as it gets — people can continue to benefit from the questions. It may happen that the scoring stops working, but then you either fix it or just remove the inline answers (or ask a friend to help you, if you have really good memory or an inclination for cheating), print it on (e-)paper and score it yourself when you're done. The point is not to get x% right, the point is finding areas that you need to train more.

I'm really curious to see how your "just works" solution using well documented PHP, MySQL/PostgreSQL and cached mimetex.cgi will pan out. What happens when the system is upgraded and some component breaks? What happens if the university hosting the project suddenly decides that all content should be kept in a CMS (as my university did a couple of years ago)? Don't get me wrong, I think it'd be great if we (are you willing to write the code?) got something like that up and running, but who is going to maintain it in the long run?

 **On multiple choice questions in mathematics « What's new** · 14 December 2008

[...] format is underway (see this server-side demo and this javascript/wiki demo, as well as the discussion here), I thought it would be a good time to collect my own personal opinions and thoughts regarding how [...]

 **Daphne** · 17 December 2008

This is somthing I have been looking for a long time. Thanks!!!

 **Audrius Meskauskas** · 23 May 2011

Dear Prof Terence Tao,
One year ago I have started the Ultrastudio.org project with the goal to collect and revive numerous open source Java applets, in part to support IcedTea where many of my friends are involved, in part with the goal to save some free code that may be very valuable as often requires competence outside the programming area. My collection currently contains about 100 applets, including 18 projects that I wrote myself, some of my own research projects.
I have more than ten years of Java programming experience and also have been a member of top importance Free software projects (such as GNU Classpath). I have already brought back to runnable condition, fixed and in some cases significantly extended a lot of open source applets that I host on my site. There were applets that needed to rewrite all GUI to drop code dependences on some ages forgotten "user friendly" GUI designer.
I see your Java applet collection as a very valuable resource on that I am ready to work seriously. It would be a great honor for me to receive from you this donation.
With respect
Audrius

 **audriusaeth** · 13 June 2011

The upgraded version of the first six Java applets about the Complex Plane is now rewritten as standalone Java application (also can still run as an applet if required). The stand alone Java application .jar can be downloaded at http://ultrastudio.org/upload/complex-13151320.jar and the source code can be downloaded http://ultrastudio.org/upload/ComplexPlane-1.6-13151542.zip .

 **Old and new apps, via modern coding agents | What's new** · 11 July 2026

[...] I have been interested in machine-assisted ways to do and teach mathematics from as far back as 1999, when I started coding several applets in Java 1.0, both for my complex analysis and linear algebra courses, to visualize various mathematical objects I was interested in (such as honeycombs or Besicovitch sets). This was moderately successful; but the applets were time-consuming to program, and eventually the standards for web pages stopped supporting this version of Java, and the applets became non-functional. [...]